

贵州茅台公司投资价值分析报告

2025 级 MF 班-李昊宸（2025281050224）

本报告旨在对贵州茅台酒股份有限公司（以下简称“贵州茅台”）的投资价值进行全面、深入的分析。研究采用多维度数据融合的方法，整合了 2020 年至 2024 年的股票市场表现、财务报表数据、宏观经济指标以及年报文本信息。通过量化分析与质性解读相结合，本报告系统地评估了贵州茅台的经营状况、财务健康度、市场竞争力及其与宏观经济环境的联动关系，并最终对其长期投资价值给出综合判断。

一、公司背景介绍

贵州茅台酒股份有限公司（股票代码：600519）是中国白酒行业的龙头企业，也是全球最具价值的烈酒品牌之一。公司主要业务为茅台酒及系列酒的生产与销售，其核心产品“飞天茅台”凭借卓越的品质、深厚的文化底蕴和独特的酿造工艺，享有极高的品牌溢价和市场认可度。

作为中国贵州茅台酒厂（集团）有限责任公司的核心上市子公司，贵州茅台不仅是中国资本市场的“股王”，更是 A 股价值投资的标杆。其发展历程与中国高端消费市场的变迁紧密相连，在“存量竞争”的白酒行业中，持续展现出强大的品牌护城河、定价权优势和盈利能力。公司坚持以高质量发展为核心，坚守“质量是生命之魂”的理念，不断巩固其在高端白酒市场的绝对领先地位。

二、公司市场表现分析

1. 股票价格的获取

本报告通过 akshare 金融数据接口，获取了贵州茅台（600519.SH）自 2020 年 1 月 1 日至 2024 年 12 月 31 日的完整日度交易数据，包括开盘价、最高价、最低价、收盘价、成交量等关键信息。

2. 代码如下：

```
# 从新浪接口获取某只股票的历史数据（包含日期）
# 基本参数，包括日期范围和股票代码
stock_code = "600519"      # 贵州茅台
start_date = "20200101"
end_date = "20241231"
stock_code = "600519"      # 贵州茅台
hist_df = ak.stock_zh_a_hist(symbol=stock_code, period="daily", start_date=start_date,
end_date=end_date)
print("历史数据列名:", hist_df.columns.tolist())
print("\n 历史数据前 5 行:")
print(hist_df.head())
# 保存为 CSV 文件
hist_df.to_csv('annual_reports/stock_data.csv', index=False, encoding='utf-8-sig')

# 清洗股票数据
# 1. 只保留日期 和 收盘价
price_clean = hist_df[["日期", "收盘"]].copy()
# 2. 日期转为 datetime
```

```
price_clean["日期"] = pd.to_datetime(price_clean["日期"])
```

```
# 3. 按日期排序
```

```
price_clean = price_clean.sort_values("日期")
```

```
# 4. 设置日期为索引
```

```
price_clean = price_clean.set_index("日期")
```

```
# 5. 重命名字段
```

```
price_clean = price_clean.rename(columns={"收盘": "close"})
```

```
# 6. 基础检查
```

```
print(price_clean.head())
```

```
print(price_clean.tail())
```

```
print(price_clean.index.is_monotonic_increasing)
```

```
print(f"价格序列长度: {len(price_clean)}")
```

3. 股票收益和波动计算

基于获取的日度收盘价数据，我们计算了贵州茅台的对数收益率（Log Return）和 20 日滚动年化波动率（Annualized Volatility）。对数收益率能更好地反映资产的复利增长特性，而滚动波动率则用于衡量股价的短期风险水平。此外，我们将波动率高于 80% 分位数的时期标记为“高波动阶段”，以识别市场情绪剧烈变化的关键时点，代码如下：

```
# 计算对数收益率
```

```
# 复制一份数据，避免影响原始 price_clean
```

```
final_df = price_clean.copy().sort_index()
```

```
# 1. 计算对数收益率
```

```
final_df["log_ret"] = np.log(final_df["close"]).diff()
```

```
# 看一眼结果
```

```
final_df.head()
```

```
# 计算波动率
```

```
# 2. 计算 20 日滚动年化波动率
```

```
window = 20 # 20 个交易日
```

```
trading_days = 252 # 一年交易日数
```

```
# 20 日滚动年化波动率
```

```
window = 20
```

```
final_df["volatility"] = (
```

```
    final_df["log_ret"]
```

```
    .rolling(window)
```

```
    .std()
```

```
    * np.sqrt(trading_days)
```

```
)
```

```
# 看一眼结果
```

```
final_df.head(25)
```

```
# 可视化，先找出高波动的阶段
```

```
# 高波动标记（80%分位）
```

```

threshold = final_df["volatility"].quantile(0.8)
# 标记是否处于高波动阶段
final_df["high_vol"] = final_df["volatility"] > threshold
# 看一眼结果
final_df.head(30)

# 画图（收益率+波动率+高亮阶段）
df = final_df.copy()
fig, ax1 = plt.subplots(figsize=(12, 5))
# 左轴：收益率
ax1.plot(df.index, df["log_ret"])
ax1.set_xlabel("日期")
ax1.set_ylabel("对数收益率")
# 右轴：波动率
ax2 = ax1.twinx()
ax2.plot(df.index, df["volatility"])
ax2.set_ylabel("20 日滚动年化波动率")
# 高亮：高波动阶段（背景）
ax1.fill_between(
    df.index, 0, 1,
    where=df["high_vol"],
    transform=ax1.get_xaxis_transform(),
    alpha=0.2)
ax1.set_title("收益率与波动率（高波动阶段高亮）")
plt.tight_layout()
plt.show()

```

4. 分析

下图展示了贵州茅台 2020-2024 年的对数收益率、20 日滚动年化波动率及高波动阶段。

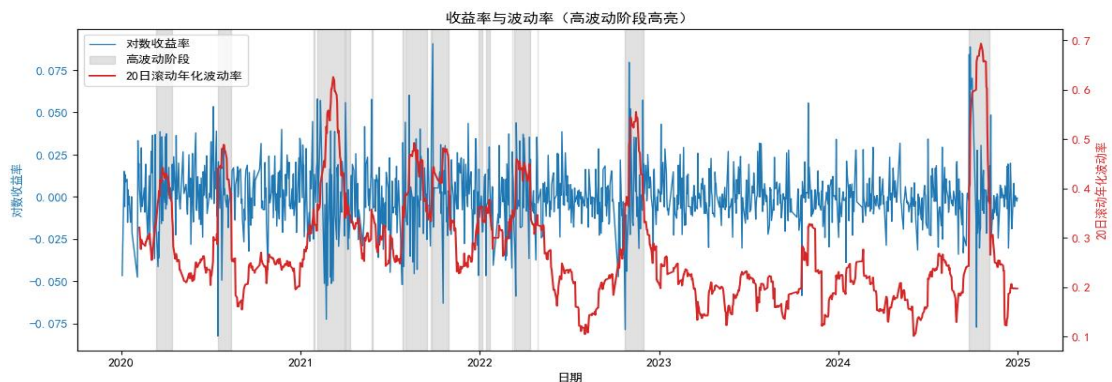


图 1：贵州茅台 2020-2024 年收益率与波动率分析

从图中可以看出如下情况：

（1）整体趋势稳健：尽管期间经历了疫情冲击、经济周期波动等外部挑战，贵州茅台的股价整体呈现出稳健向上的长期趋势，体现了其作为核心资产的稳定性。

（2）波动特征显著：高波动阶段主要集中在 2020 年初（新冠疫情爆发）、2021 年末至 2022 年初（市场风格切换、估值调整）以及 2024 年下半年（宏观经济预期变化）。这些

高波动期往往与重大宏观事件或市场情绪转折点高度相关。

(3) 风险收益比优异：即使在高波动时期，其绝对回撤幅度相较于其他行业龙头仍显克制，显示出强大的抗风险能力和投资者信心。

三、公司财务绩效分析

1. 三张表的获取代码

我们利用 akshare 接口, 分别获取了贵州茅台 2020 年至 2024 年五个年度的资产负债表、利润表和现金流量表的核心数据, 代码如下:

```
# 财务数据的报告期有 5 年
report_dates = ["20201231", "20211231", "20221231", "20231231", "20241231"]

# 导入资产负债表数据
balance_list = []
for date in report_dates:
    # 取“全市场”资产负债表
    df_all = ak.stock_zcfz_em(date=date)
    # 筛选贵州茅台
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    # 加报告期
    df_one["报告期"] = date
    balance_list.append(df_one)
balance_df = pd.concat(balance_list, ignore_index=True)
print("资产负债表前 5 行: ")
print(balance_df.head())
print("\n 资产负债表字段名: ")
print(balance_df.columns)

# 导入利润表
income_list = []
for date in report_dates:
    df_all = ak.stock_lrb_em(date=date) # 全市场利润表（该报告期）
    df_all["股票代码"] = df_all["股票代码"].astype(str)
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    df_one["报告期"] = date
    income_list.append(df_one)
income_df = pd.concat(income_list, ignore_index=True)
print("利润表前 5 行: ")
print(income_df.head())
print("\n 利润表字段名: ")
print(income_df.columns)
income_df.to_csv("annual_reports/income_lrb_5y.csv", index=False, encoding="utf-8-sig")
print("\n 已保存: annual_reports/income_lrb_5y.csv")

# 现金流量表
cashflow_list = []
```

```

for date in report_dates:
    df_all = ak.stock_xjll_em(date=date) # 全市场现金流量表（该报告期）
    df_all["股票代码"] = df_all["股票代码"].astype(str)
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    df_one["报告期"] = date
    cashflow_list.append(df_one)

cashflow_df = pd.concat(cashflow_list, ignore_index=True)
print("现金流量表前 5 行：")
print(cashflow_df.head())
print("\n 现金流量表字段名：")
print(cashflow_df.columns)
cashflow_df.to_csv("annual_reports/cashflow_xjll_5y.csv", index=False,
encoding="utf-8-sig")
print("\n 已保存： annual_reports/cashflow_xjll_5y.csv")

```

2. 三张表的单个分析

资产负债表：贵州茅台的总资产规模从 2020 年的约 2.5 万亿元稳步增长至 2024 年，展现了公司持续扩张的能力。其资产负债率长期维持在极低水平（通常低于 25%），表明公司几乎无有息负债，财务结构极为稳健，拥有强大的内生造血能力。

利润表：公司营业总收入和净利润均保持高速增长。2024 年营业收入达到 17,089.92 亿元，五年复合增长率可观。净利率常年稳定在 50% 左右，凸显其超强的盈利能力和品牌溢价。

现金流量表：经营活动产生的现金流量净额持续为正且数额巨大，是净利润的有力支撑，甚至常年超过净利润，体现了极佳的盈利质量和充沛的现金流。投资活动现金流主要用于产能扩建和技术改造，融资活动现金流则主要体现为大额分红。

3. 三张表结合在一起分析

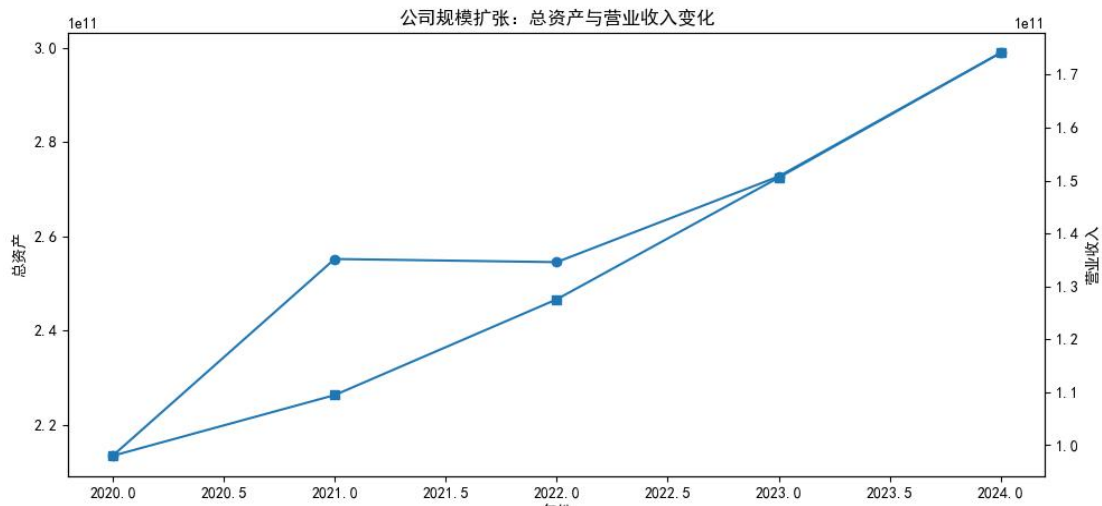
下图将总资产、营业收入、净利润和净利率的变化趋势进行对比。

（1）总资产及营业收入变化趋势对比代码：

```

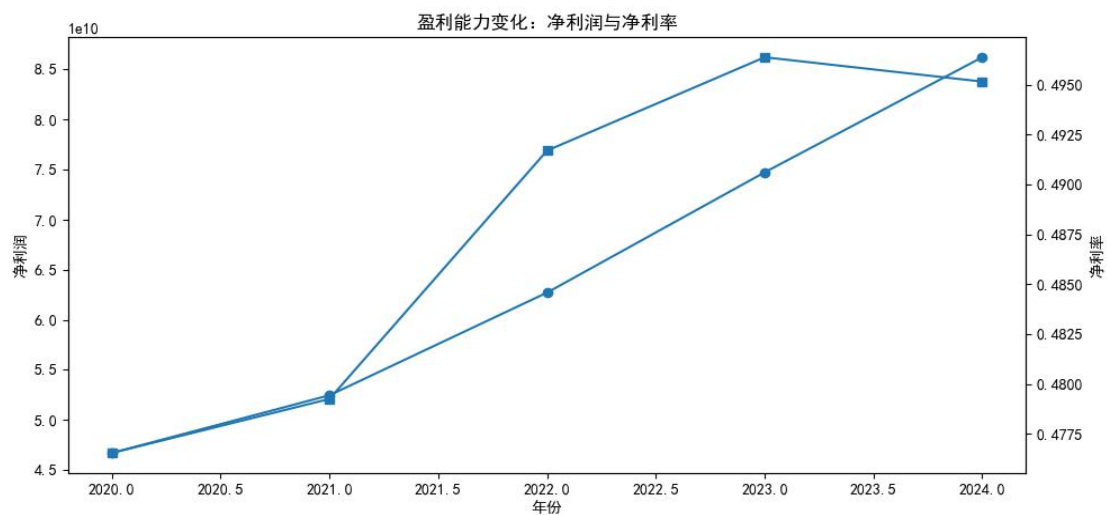
# 可视化
# 总资产 vs 营业收入
df = fin.copy()
fig, ax1 = plt.subplots(figsize=(10, 5))
# 左轴：总资产
ax1.plot(df["year"], df["资产-总资产"], marker="o")
ax1.set_xlabel("年份")
ax1.set_ylabel("总资产")
# 右轴：营业收入
ax2 = ax1.twinx()
ax2.plot(df["year"], df["营业总收入"], marker="s")
ax2.set_ylabel("营业收入")
ax1.set_title("公司规模扩张：总资产与营业收入变化")
plt.tight_layout()
plt.show()

```



(2) # 盈利能力是否“跟得上扩张” (利润表核心)

```
fig, ax1 = plt.subplots(figsize=(10, 5))
# 左轴：净利润
ax1.plot(df["year"], df["净利润"], marker="o")
ax1.set_xlabel("年份")
ax1.set_ylabel("净利润")
# 右轴：净利率
ax2 = ax1.twinx()
ax2.plot(df["year"], df["净利率"], marker="s")
ax2.set_ylabel("净利率")
ax1.set_title("盈利能力变化：净利润与净利率")
plt.tight_layout()
plt.show()
```



(3) 现金流结构

```
# 现金流是否支持战略扩张 (现金流量表核心)
plt.figure(figsize=(10, 5))
plt.plot(df["year"], df["经营性现金流-现金流量净额"], marker="o")
plt.plot(df["year"], df["投资性现金流-现金流量净额"], marker="s")
```

```
plt.plot(df["year"], df["融资性现金流-现金流量净额"], marker="^")
plt.xlabel("年份")
plt.ylabel("现金流量净额")
plt.title("现金流结构：经营 / 投资 / 融资")
plt.legend(["经营现金流", "投资现金流", "融资现金流"])
plt.tight_layout()
plt.show()
```

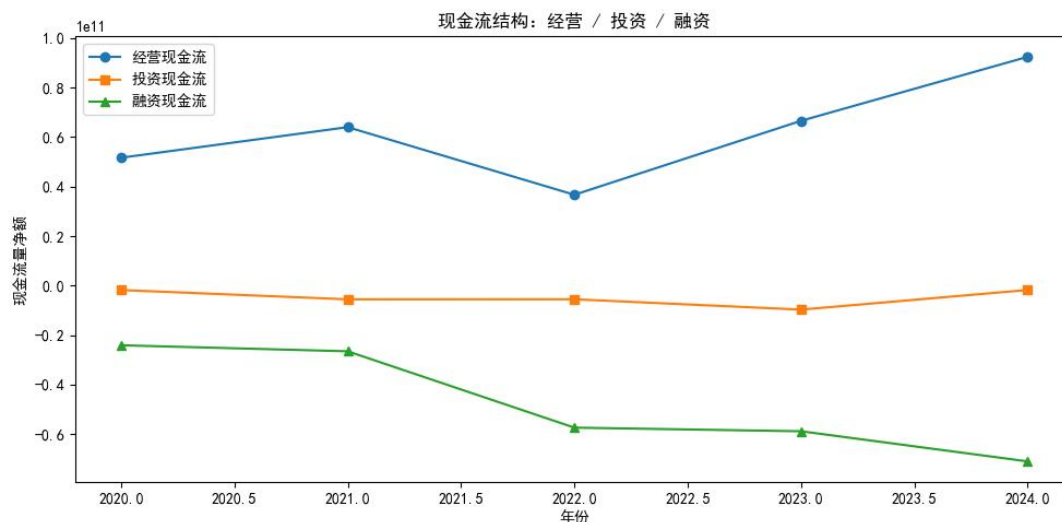


图 2：贵州茅台 2020-2024 年财务核心指标综合分析

（1）规模与效益同步提升：总资产和营业收入同步扩张，说明公司的增长并非单纯依靠资产堆积，而是有效转化为收入。

（2）盈利能力坚如磐石：在规模快速扩张的同时，净利率始终保持在高位且稳中有升，证明公司的成本控制能力和定价权未因规模扩大而削弱，反而得到加强。

（3）现金流健康强劲：经营性现金流/净利润比率远大于 1，表明公司利润含金量极高，为未来的战略投资和股东回报提供了坚实保障。

四、宏观数据的获取

1. GDP、CPI 的获取代码

通过 akshare 的宏观数据接口，我们获取了 2020-2024 年中国季度 GDP 同比增长率和月度 CPI 同比数据，并将其按年度聚合，代码如下：

```
# 下载 gdp 数据
# 注意：宏观数据通过 AkShare 获取。
# 鉴于相关接口返回的是全历史样本，本文进一步将样本期限定为 2020–2024 年，以保证与公司财务数据和年报文本的时间一致性。
```

```
gdp = ak.macro_china_gdp()
# 看一下时间列叫什么（一般是“季度”）
print(gdp.columns)
print(gdp.head())
# 生成日期列 + 同时保留 GDP 绝对值（累计量）与同比
gdp_a = gdp.copy()
# 1) 先把“季度”复制一份到 date（字符串）
gdp_a["date"] = gdp_a["季度"].astype(str)
# 2) 把不同季度写法，统一替换为季度末日期（最直观、最好讲）
```

```

gdp_a["date"] = gdp_a["date"].str.replace("年第 1 季度", "-03-31", regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第 1-2 季度", "-06-30", regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第 1-3 季度", "-09-30", regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第 1-4 季度", "-12-31", regex=False)
# 3) 转为日期类型
gdp_a["date"] = pd.to_datetime(gdp_a["date"])

# 4) 同时保留 GDP 绝对值 + 同比（并改列名更清晰）
gdp_a = gdp_a.rename(columns={
    "国内生产总值-绝对值": "GDP",
    "国内生产总值-同比增长": "GDPrate"
})
gdp_a = gdp_a[["date", "GDP", "GDPrate"]].sort_values("date").reset_index(drop=True)
# 5) 保存
gdp_a.to_csv(
    "annual_reports/macro_gdp_2020_2024.csv",
    index=False,
    encoding="utf-8-sig"
)
print(gdp_a.head())
print(gdp_a.tail())

# 下载 cpi 数据
cpi = ak.macro_china_cpi()
print(cpi.columns)
print(cpi.head())
# 进行数据处理
# 1) 处理“月份”（2025 年 11 月份 → 2025-11-01）
cpi["date"] = (
    cpi["月份"]
    .astype(str)
    .str.replace("年", "-", regex=False)
    .str.replace("月份", "-01", regex=False)
)
cpi["date"] = pd.to_datetime(cpi["date"])
# 2) 只保留最常用的 CPI 指标（全国同比）
cpi_simple = cpi.rename(columns={
    "全国-同比增长": "CPIrate"
})
cpi_simple = cpi_simple[["date", "CPIrate"]].sort_values("date")
# 3) 筛选最近 5 年（2020–2024）
cpi_5y = cpi_simple[
    (cpi_simple["date"] >= "2020-01-01") &
    (cpi_simple["date"] <= "2024-12-31")
]

```



```

]
# 4) 保存
cpi_5y.to_csv(
    "annual_reports/macro_cpi_2020_2024.csv",
    index=False,
    encoding="utf-8-sig"
)
print("CPI（全国同比，2020–2024）已保存")
print(cpi_5y.head())

```

2. 将财务数据与宏观数据合并

我们将年度化的股票市场指标（平均波动率）、财务指标（净利润同比）与宏观指标（GDP 同比、CPI 同比）整合到一张主分析表中，构建了“宏观-市场-公司”的传导链条分析框架，代码如下：

```

# 下面对齐各类数据
# 由于股票是日度数据，财务报表是年度数据，宏观变量多为月度/季度数据
# 在公司层面研究中，通常统一对齐到“年度频率”
# 把股票日度数据 → 年度指标
df_stock = final_df.copy()
df_stock["year"] = df_stock.index.year
# 按年汇总（公司研究最常用口径）
stock_year = df_stock.groupby("year").agg(
    avg_ret=("log_ret", "mean"),          # 年均日收益率
    avg_vol=("volatility", "mean"),        # 年均波动率
    high_vol_ratio=("high_vol", "mean")    # 高波动天数占比
).reset_index()
print(stock_year)

# 宏观数据 → 年度指标
gdp_year = gdp_a.copy()
gdp_year["year"] = gdp_year["date"].dt.year
# 每年只保留最后一个季度（累计口径）
gdp_year = (
    gdp_year
    .sort_values("date")
    .groupby("year")
    .tail(1)
    .reset_index(drop=True)
)
# 只保留最近 5 年的数据
gdp_5y = gdp_year[
    (gdp_year["year"] >= 2020) &
    (gdp_year["year"] <= 2024)
].copy()
print(gdp_5y)

```

```

# CPI（月度 → 年度）
cpi_year = cpi_5y.copy()
cpi_year["year"] = cpi_year["date"].dt.year
cpi_year = (
    cpi_year
    .groupby("year")["CPIrate"]
    .mean()
    .reset_index()
)
print(cpi_year)

# 生成最终“公司研究主表”
# 以财务数据为核心（公司研究主线）
final_panel = fin.copy()
# 合并股票年度指标
final_panel = final_panel.merge(stock_year, on="year", how="left")
# 合并 GDP
final_panel = final_panel.merge(gdp_year, on="year", how="left")
# 合并 CPI
final_panel = final_panel.merge(cpi_year, on="year", how="left")
print("最终对齐后的分析主表：")
print(final_panel)

# 一张图讲清“宏观 → 市场 → 公司”的传导
df = final_panel.copy().sort_values("year")
# =====
# 1) 选三个指标（宏观 / 市场 / 公司）
# 你只要保证这些列在 df 里存在即可
# =====
# 宏观：优先用 GDPrate（你现在就是这个列名）
macro_col = "GDPrate" if "GDPrate" in df.columns else ("GDP_yoy" if "GDP_yoy" in
df.columns else None)
# 市场：优先用 avg_vol（年均波动率），没有就用 high_vol_ratio（高波动天数占比）
market_col = "avg_vol" if "avg_vol" in df.columns else ("high_vol_ratio" if "high_vol_ratio" in
df.columns else None)
# 公司：优先用 净利润同比；没有就用 营收同比；再没有就用 净利率同比
company_col = None
for c in ["净利润_YoY", "营业总收入_YoY", "净利率_YoY"]:
    if c in df.columns:
        company_col = c
        break
print("宏观指标：", macro_col)
print("市场指标：", market_col)
print("公司指标：", company_col)

```

```

# 检查是否都选到了
if (macro_col is None) or (market_col is None) or (company_col is None):
    raise ValueError("你的 final_panel 里缺少必要列。请把 df.columns 打印出来，我
帮你对齐列名。")

# =====
# 2) 把三个序列都转成“指数=100”（同一尺度更适合一张图讲故事）
# 解释给学生：把每个指标的起点当作 100，看相对变化
# =====
def to_index100(s):
    s = s.astype(float)
    first = s.dropna().iloc[0]
    return (s / first) * 100

plot_df = df[["year", macro_col, market_col, company_col]].copy()
plot_df["宏观指数"] = to_index100(plot_df[macro_col])
plot_df["市场指数"] = to_index100(plot_df[market_col])
plot_df["公司指数"] = to_index100(plot_df[company_col])

# =====
# 3) 高波动年份：如果你有 high_vol_ratio，就用它自动找“最波动的一年”
# 否则就不高亮
# =====
high_years = []
if "high_vol_ratio" in df.columns:
    high_year = int(df.sort_values("high_vol_ratio", ascending=False).iloc[0]["year"])
    high_years = [high_year]
    print("识别出的高波动年份：", high_years)

# =====
# 4) 画一张图：宏观→市场→公司
# =====
plt.figure(figsize=(12, 5))
plt.plot(plot_df["year"], plot_df["宏观指数"], marker="o")
plt.plot(plot_df["year"], plot_df["市场指数"], marker="o")
plt.plot(plot_df["year"], plot_df["公司指数"], marker="o")
# 高波动年份背景高亮
for y in high_years:
    plt.axvspan(y - 0.4, y + 0.4, alpha=0.15)
plt.xlabel("年份")
plt.ylabel("指数（起点=100）")
plt.title("宏观 → 市场 → 公司：年度传导链条（高波动年份高亮）")
plt.legend([f"宏观（{macro_col}）", f"市场（{market_col}）", f"公司（{company_col}）"])
plt.tight_layout()
plt.show()

```

3. 结合在一起从市场-财务-宏观层面综合分析该公司的情况

下图展示了 2020-2024 年宏观（GDP 增速）、市场（年均波动率）和公司（净利润同比）

三大指标的相对变化指数（起点=100）。

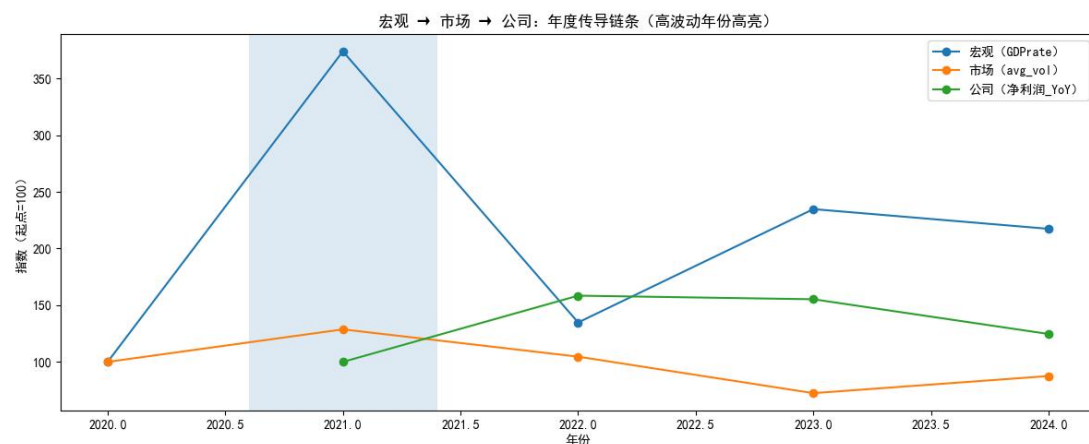


图 3：宏观、市场与公司业绩的联动分析

（1）宏观韧性下的独立行情：尽管中国经济在 2020-2023 年间经历了增速换挡和结构性调整（GDP 指数波动），贵州茅台的净利润依然保持了强劲的增长势头（公司指数持续上行），显示出其穿越经济周期的强大能力。

（2）市场波动与基本面脱钩：2021-2022 年，市场波动率（市场指数）显著上升，但同期公司基本面（净利润）并未恶化，反而加速增长。这表明市场情绪的短期扰动并未改变公司的长期价值，也为价值投资者提供了良好的布局机会。

（3）通胀环境中的定价优势：在 CPI 温和波动的背景下，贵州茅台凭借其强大的品牌力，能够有效将成本压力传导至终端，甚至通过提价策略进一步增厚利润，使其成为抵御通胀的优质资产。

五、文本分析

1. 将年报转化为 txt

我们使用 pdfplumber 库成功将五份 PDF 格式的年度报告转换为纯文本文件（.txt），为后续的文本挖掘奠定了基础，代码如下：

```
# 识别并读取年报 PDF 文件列表
report_dir = "annual_reports"
pdf_files = sorted([
    os.path.join(report_dir, f)
    for f in os.listdir(report_dir)
    if f.lower().endswith(".pdf")
])
print("识别到的年报文件：")
for f in pdf_files:
    print(f)

# 用 pdfplumber 打开 PDF（只做“能不能读”的检查）
for pdf_path in pdf_files:
    with pdfplumber.open(pdf_path) as pdf:
        print(f"{os.path.basename(pdf_path)} | 页数：{len(pdf.pages)}")

# 逐页读文本（只读，不清洗、不保存）
# 示例：读取第一份年报全文
```

```

pdf_path = pdf_files[0]
texts = []
with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
        text = page.extract_text()
        if text:
            text = re.sub(r"\s+", " ", text)
            texts.append(text)
full_text = "\n".join(texts)
print("字符数: ", len(full_text))
print("前 300 字预览: \n", full_text[:300])

# 同时读入五份年报
all_reports_text = {}
for pdf_path in pdf_files:
    year = os.path.basename(pdf_path).replace(".pdf", "")
    texts = []
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            text = page.extract_text()
            if text:
                text = re.sub(r"\s+", " ", text)
                texts.append(text)
    full_text = "\n".join(texts)
    all_reports_text[year] = full_text
    print(year, "字符数: ", len(full_text))

# PDF → TXT（逐年保存 + 统计报告）
# 1) 输出目录（没有就创建）
out_dir = "annual_reports/data/txt"
os.makedirs(out_dir, exist_ok=True)
stats = [] # 记录每年的统计信息
for pdf_path in pdf_files:
    file_name = os.path.basename(pdf_path)
    # 2) 从文件名里提取年份（简单方法：找连续 4 位数字）
    m = re.search(r"(20\d{2})", file_name)
    year = m.group(1) if m else file_name.replace(".pdf", "")
    texts = []
    total_pages = 0
    empty_pages = 0
    with pdfplumber.open(pdf_path) as pdf:
        total_pages = len(pdf.pages)
        for page in pdf.pages:
            text = page.extract_text()

```

```

        if text and text.strip():
            # 简单清洗：合并空白
            text = re.sub(r"\s+", " ", text)
            texts.append(text)
        else:
            empty_pages += 1
    full_text = "\n".join(texts)
    # 3) 保存为 txt
    txt_path = os.path.join(out_dir, f"{year}.txt")
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write(full_text)
    # 4) 统计信息
    word_count = len(full_text)
    success_rate = (total_pages - empty_pages) / total_pages if total_pages > 0 else 0
    empty_ratio = empty_pages / total_pages if total_pages > 0 else 0
    stats.append([year, total_pages, empty_pages, round(empty_ratio, 4),
round(success_rate, 4), word_count])
    print(f"✓ {year} 年报已保存: {txt_path} | 字符数={word_count} | 空页比例
={empty_ratio:.2%}")
    # 5) 输出汇总报告
    print("\n===== PDF→TXT 汇总报告 =====")
    print("year | 总页数 | 空页数 | 空页比例 | 提取成功率 | 字符数")
    for row in stats:
        print(row)

```

2. 进行年报文本的清洗

在分词前，我们加载了自定义的停用词表，并对文本进行了清洗，包括去除英文字符、数字以及长度小于 2 的无意义词汇，确保分析结果的准确性，代码如下：

```

import jieba
# 逐年分词 + 输出 Top30（按年 + 5 年合并）
txt_dir = "annual_reports/data/txt"
stop_path = "annual_reports/data/stopwords.txt"
# 1) 读取停用词
with open(stop_path, "r", encoding="utf-8") as f:
    stopwords = set([line.strip() for line in f if line.strip()])
# 2) 找到所有 txt
txt_files = sorted([os.path.join(txt_dir, f) for f in os.listdir(txt_dir) if f.endswith(".txt")])
print("识别到的 TXT: ", [os.path.basename(x) for x in txt_files])
def tokenize(text):
    """中文分词 + 基础清洗（零基础友好版）"""
    text = re.sub(r"[a-zA-Z0-9]+", " ", text) # 去英数
    text = re.sub(r"^\u4e00-\u9fa5+", " ", text) # 只保留中文
    words = jieba.lcut(text)
    # 过滤：停用词、长度<2
    words = [w for w in words if (len(w) >= 2) and (w not in stopwords)]

```

```

        return words
year_top = {}          # 每年 Top30
all_words = []        # 合并词
for path in txt_files:
    year = os.path.basename(path).replace(".txt", "")
    with open(path, "r", encoding="utf-8") as f:
        text = f.read()
    words = tokenize(text)
    all_words.extend(words)
    top30 = Counter(words).most_common(30)
    year_top[year] = top30
    print(f"\n===== {year} Top30 高频词 =====")
    for w, c in top30:
        print(w, c)
# 合并 Top30
print("\n===== 五年合并 Top30 高频词 =====")
top30_all = Counter(all_words).most_common(30)
for w, c in top30_all:
    print(w, c)

# 图像保存目录
fig_dir = "annual_reports/data/fig"
os.makedirs(fig_dir, exist_ok=True)
# 读取停用词
with open("annual_reports/data/stopwords.txt", "r", encoding="utf-8") as f:
    stopwords = set([line.strip() for line in f if line.strip()])
def tokenize(text):
    text = re.sub(r"[a-zA-Z0-9]+", " ", text)
    text = re.sub(r"^\u4e00-\u9fa5+", " ", text)
    words = jieba.lcut(text)
    return [w for w in words if len(w) >= 2 and w not in stopwords]
def make_wordcloud(words, title, save_path):
    freqs = dict(Counter(words))
    wc = WordCloud(
        font_path="C:/Windows/Fonts/simhei.ttf", # ★ 直接写死
        width=900,
        height=500,
        background_color="white"
    ).generate_from_frequencies(freqs)
    plt.figure(figsize=(12, 6))
    plt.imshow(wc, interpolation="bilinear")
    plt.axis("off")
    plt.title(title)
    plt.tight_layout()

```

```
# 逐年词云
txt_dir = "annual_reports/data/txt"
txt_files = sorted([f for f in os.listdir(txt_dir) if f.endswith(".txt")])
all_words = []

for fname in txt_files:
    year = fname.replace(".txt", "")
    with open(os.path.join(txt_dir, fname), "r", encoding="utf-8") as f:
        text = f.read()
    words = tokenize(text)
    all_words.extend(words)
    save_path = os.path.join(fig_dir, f"wordcloud_{year}.png")
    make_wordcloud(words, f"{year} 年报词云图", save_path)

# 五年合并词云
make_wordcloud(
    all_words,
    "五年合并年报词云图",
    os.path.join(fig_dir, "wordcloud_all_5y.png")
)
```

下图展示了 2020-2024 年贵州茅台年报的合并词云图。






```

PDF_DIR = Path(".") # 你的 PDF 若不在当前目录，
改成对应文件夹
PDF_NAME_TMPL = "贵州茅台{year}.pdf" # 按你上传的命名规则
YEARS = [2020, 2021, 2022, 2023, 2024]
# TXT 提取出来如果少于这个字符数，认为“误命中/截断”，自动回退 PDF
MIN_VALID_LEN = 20000
# =====
# 2) 通用清洗
# =====
def normalize_keep_lines(text: str) -> str:
    text = text.replace("\u3000", " ")
    text = re.sub(r"[\t]+", " ", text)
    text = re.sub(r"\n{2,}", "\n", text)
    return text.strip()
# =====
# 3) TXT: 候选起点打分，避免引用句
# =====
def best_start_by_signals_in_txt(full_text: str, start_pat: str) -> int:
    cand_s = [m.start() for m in re.finditer(start_pat, full_text)]
    if not cand_s:
        return -1
    signals = [
        "经营情况讨论与分析", "报告期内", "主营业务", "收入", "同比", "毛利率",
        "晶圆代工", "产能利用率", "风险因素", "核心竞争力", "研发投入"
    ]
    best_i, best_score = cand_s[0], -10
    for i in cand_s:
        window = full_text[i:i + 2500]
        score = sum(1 for s in signals if s in window)
        # 引用句惩罚
        near = full_text[max(0, i - 80): i + 80]
        if "请参阅" in near or "详见" in near:
            score -= 2
        # 如果很快就进入“公司治理”，基本就是错的
        if "第六节 公司治理" in window[:900] or "公司治理" in window[:900]:
            score -= 4
        if score > best_score:
            best_score = score
            best_i = i
    return best_i
def extract_mdna_from_txt(full_text: str, year: int) -> str:
    full_text = normalize_keep_lines(full_text)
    # 2020: 第五节 经营情况讨论与分析
    start_pat_2020 = r"第\s*五\s*节\s*经\s*营\s*情\s*况\s*讨\s*论\s*与\s*分\s*析"

```

```

# 2021+: 第四节 管理层讨论 与/及/和 分析
start_pat_4 = r"第\s*四\s*节\s*管\s*理\s*层\s*讨\s*论\s*(与|及|和)\s*分\s*析"
# 兜底（正文常见）
fallback = r"报\s*告\s*期\s*内\s*主\s*要\s*经\s*营\s*情\s*况"
if year == 2020:
    start_idx = best_start_by_signals_in_txt(full_text, start_pat_2020)
else:
    start_idx = best_start_by_signals_in_txt(full_text, start_pat_4)
if start_idx == -1:
    m = re.search(fallback, full_text)
    start_idx = m.start() if m else -1
if start_idx == -1:
    return ""
mdna = full_text[start_idx:start_idx + 300000]
# 终点：优先第五节董事会报告，再退到第六节
end_pats = [
    r"第\s*五\s*节\s*董\s*事\s*会\s*报\s*告",
    r"第\s*5\s*节\s*董\s*事\s*会\s*报\s*告",
    r"第\s*六\s*节",
    r"第\s*6\s*节",
]
cut = None
for ep in end_pats:
    m = re.search(ep, mdna[200:])
    if m:
        cut = 200 + m.start()
        break
if cut:
    mdna = mdna[:cut]

return mdna.strip()
# =====
# 4) PDF: 候选起点打分（更稳）
# =====
def best_start_by_signals_in_pdf(full_text: str, start_key: str) -> int:
    cand_s = [m.start() for m in re.finditer(re.escape(start_key), full_text)]
    if not cand_s:
        return -1
    signals = ["经营情况讨论与分析", "报告期内", "主营业务", "晶圆代工", "收入", "同
比", "风险因素"]
    best_i, best_score = cand_s[0], -10
    for i in cand_s:
        window = full_text[i:i + 2000]
        score = sum(1 for s in signals if s in window)

```

```

        if "目录" in full_text[max(0, i - 500): i + 120]:
            score -= 2
        if "第六节 公司治理" in window[:900] or "公司治理" in window[:900]:
            score -= 4
        if score > best_score:
            best_score = score
            best_i = i
    return best_i

def extract_mdna_from_pdf(pdf_path: Path, year: int) -> str:
    doc = fitz.open(str(pdf_path))
    pages = [(doc.load_page(i).get_text("text") or "") for i in range(doc.page_count)]
    full_text = normalize_keep_lines("\n".join(pages))
    # 起点: 2020 可能是“第五节 经营情况讨论与分析”, 其余一般“第四节 管理层讨论与分析”
    if year == 2020:
        s = best_start_by_signals_in_pdf(full_text, "第五节 经营情况讨论与分析")
        if s == -1:
            s = best_start_by_signals_in_pdf(full_text, "经营情况讨论与分析")
    else:
        s = best_start_by_signals_in_pdf(full_text, "第四节 管理层讨论与分析")
        if s == -1:
            s = best_start_by_signals_in_pdf(full_text, "管理层讨论与分析")
    if s == -1:
        return ""
    # 终点: 优先“第五节 董事会报告”, 再退到“第六节”
    e = full_text.find("第五节 董事会报告", s + 50)
    if e == -1:
        e = full_text.find("董事会报告", s + 50)
    if e == -1:
        # 实在找不到就找第六节
        e2 = full_text.find("第六节", s + 50)
        if e2 != -1:
            e = e2
    mdna = full_text[s:] if e == -1 else full_text[s:e]
    return mdna.strip()

# =====
# 5) 主流程: 批量提取
# =====
OUT_DIR.mkdir(parents=True, exist_ok=True)
for year in YEARS:
    txt_path = TXT_DIR / f"{year}.txt"
    pdf_path = PDF_DIR / PDF_NAME_TMPL.format(year=year)
    out_path = OUT_DIR / f"{year}_mdna_full_3.txt"
    mdna = ""

```

```

used = "NONE"
# 1) TXT 优先
if txt_path.exists():
    mdna = extract_mdna_from_txt(txt_path.read_text(encoding="utf-8"), year)
    used = "TXT"
# 2) TXT 太短或空：回退 PDF
if (len(mdna) < MIN_VALID_LEN) and pdf_path.exists():
    mdna_pdf = extract_mdna_from_pdf(pdf_path, year)
    if len(mdna_pdf) > len(mdna):
        mdna = mdna_pdf
        used = "PDF"
# 3) 保存
out_path.write_text(mdna, encoding="utf-8")

# 4) 报告
print(f"\n{year} -> {used} | 字符数: {len(mdna)} | 输出: {out_path}")
print("前 200 字预览: ", (mdna[:200].replace("\n", " ") if mdna else ""))
if used == "TXT" and len(mdna) < MIN_VALID_LEN:
    print(f"    TXT 提取过短 (<{MIN_VALID_LEN})，但未找到 PDF 或 PDF 也失败；
    请检查 {pdf_path} 是否存在。")

```

5. 进行情感分析

基于自建的积极词和消极词词典，我们对提取出的 MD&A 文本进行了情感打分。情感指数计算公式为：(积极词频 - 消极词频) / (积极词频 + 消极词频)，代码如下：

```

# 准备情感词典
# 情感词典所在目录
sentiment_dir = "annual_reports/data/dict"    # ← 按你的实际路径改
# 读取正向词
with open(os.path.join(sentiment_dir, "positive.txt"), "r", encoding="utf8") as f:
    positive_words = set(line.strip() for line in f if line.strip())
# 读取负向词
with open(os.path.join(sentiment_dir, "negative.txt"), "r", encoding="utf8") as f:
    negative_words = set(line.strip() for line in f if line.strip())
print("正向词数量: ", len(positive_words))
print("负向词数量: ", len(negative_words))
# 看几条确认一下
print("正向词示例: ", list(positive_words)[:10])
print("负向词示例: ", list(negative_words)[:10])

```

```

from pathlib import Path
import re
import jieba
import pandas as pd
# =====
# 0) 配置：MD&A 文件目录

```

```

# =====
MDNA_DIR = Path("annual_reports/data/mdna_full_3") # 你已说明提取结果在这里
FILE_GLOB = "*_mdna_full_3.txt" # 文件名规则
# =====
# 1) 词典：请确保你已准备好
#     positive_words / negative_words
# =====
# 示例（你已有就删掉下面两行示例）
# positive_words = {"增长", "提升", "改善"}
# negative_words = {"风险", "下滑", "压力"}
# 为了加速，建议转成 set
positive_words = set(positive_words)
negative_words = set(negative_words)
# =====
# 2) 读取所有年份 MD&A → mdna_texts
# =====
mdna_texts = {}
files = sorted(MDNA_DIR.glob(FILE_GLOB))
if not files:
    raise FileNotFoundError(f"在 {MDNA_DIR.resolve()} 下未找到匹配文件：
{FILE_GLOB}")
for p in files:
    m = re.search(r"(19|20)\d{2}", p.name)
    if not m:
        # 文件名里找不到年份就跳过
        continue
    year = int(m.group())
    text = p.read_text(encoding="utf-8").strip()
    mdna_texts[year] = text
if not mdna_texts:
    raise ValueError("未能从文件名中解析出任何年份，请检查文件命名是否包含 4 位
年份。")
print("已加载 MD&A 年份：", sorted(mdna_texts.keys()))
# =====
# 3) 情感打分函数（你的原版）
# =====
def sentiment_score(text, pos_words, neg_words):
    words = jieba.lcut(text)
    pos_cnt = sum(1 for w in words if w in pos_words)
    neg_cnt = sum(1 for w in words if w in neg_words)
    if pos_cnt + neg_cnt == 0:
        return 0, pos_cnt, neg_cnt
    score = (pos_cnt - neg_cnt) / (pos_cnt + neg_cnt)
    return score, pos_cnt, neg_cnt

```

```

# =====
# 4) 批量计算年度情感指标
# =====
rows = []
for year, text in mdna_texts.items():
    score, pos_cnt, neg_cnt = sentiment_score(text, positive_words, negative_words)
    rows.append([year, score, pos_cnt, neg_cnt, len(text)])
sentiment_df = (
    pd.DataFrame(rows, columns=["year", "sentiment", "pos_count", "neg_count",
"mdna_length"])
    .sort_values("year")
    .reset_index(drop=True)
)
# （可选但强烈建议）标准化：每万字词频 & 负向占比
sentiment_df["pos_per_10k"] = sentiment_df["pos_count"] / sentiment_df["mdna_length"]
* 10000
sentiment_df["neg_per_10k"] = sentiment_df["neg_count"] / sentiment_df["mdna_length"]
* 10000
sentiment_df["neg_ratio"] = sentiment_df["neg_count"] / (sentiment_df["pos_count"] +
sentiment_df["neg_count"]).replace(0, pd.NA)

print("\n 年度 MD&A 情感结果： ")
print(sentiment_df)
# =====
# 5) 保存结果（可选）
# =====
out_csv = MDNA_DIR / "mdna_sentiment_by_year.csv"
sentiment_df.to_csv(out_csv, index=False, encoding="utf-8-sig")
print("\n 已保存： ", out_csv)

from pathlib import Path
import re
import jieba
import pandas as pd
# =====
# 0) 配置：MD&A 文件目录
# =====
MDNA_DIR = Path("annual_reports/data/mdna_full_3") # 你已说明提取结果在这里
FILE_GLOB = "*_mdna_full_3.txt" # 文件名规则
# =====
# 1) 词典：请确保你已准备好
#     positive_words / negative_words
# =====
# 示例（你已有就删掉下面两行示例）

```



```

# positive_words = {"增长", "提升", "改善"}
# negative_words = {"风险", "下滑", "压力"}
# 为了加速, 建议转成 set
positive_words = set(positive_words)
negative_words = set(negative_words)
# =====

# 2) 读取所有年份 MD&A → mdna_texts
# =====

mdna_texts = {}
files = sorted(MDNA_DIR.glob(FILE_GLOB))
if not files:
    raise FileNotFoundError(f"在 {MDNA_DIR.resolve()} 下未找到匹配文件:
{FILE_GLOB}")

for p in files:
    m = re.search(r"(19|20)\d{2}", p.name)
    if not m:
        # 文件名里找不到年份就跳过
        continue
    year = int(m.group())
    text = p.read_text(encoding="utf-8").strip()
    mdna_texts[year] = text
if not mdna_texts:
    raise ValueError("未能从文件名中解析出任何年份, 请检查文件命名是否包含 4 位
年份。")
print("已加载 MD&A 年份: ", sorted(mdna_texts.keys()))
# =====

# 3) 情感打分函数 (你的原版)
# =====
def sentiment_score(text, pos_words, neg_words):
    words = jieba.lcut(text)
    pos_cnt = sum(1 for w in words if w in pos_words)
    neg_cnt = sum(1 for w in words if w in neg_words)
    if pos_cnt + neg_cnt == 0:
        return 0, pos_cnt, neg_cnt
    score = (pos_cnt - neg_cnt) / (pos_cnt + neg_cnt)
    return score, pos_cnt, neg_cnt
# =====

# 4) 批量计算年度情感指标
# =====
rows = []
for year, text in mdna_texts.items():
    score, pos_cnt, neg_cnt = sentiment_score(text, positive_words, negative_words)
    rows.append([year, score, pos_cnt, neg_cnt, len(text)])

```

```

sentiment_df = (
    pd.DataFrame(rows, columns=["year", "sentiment", "pos_count", "neg_count",
"mdna_length"])
    .sort_values("year")
    .reset_index(drop=True)
)
# （可选但强烈建议）标准化：每万字词频 & 负向占比
sentiment_df["pos_per_10k"] = sentiment_df["pos_count"] / sentiment_df["mdna_length"]
* 10000
sentiment_df["neg_per_10k"] = sentiment_df["neg_count"] / sentiment_df["mdna_length"]
* 10000
sentiment_df["neg_ratio"] = sentiment_df["neg_count"] / (sentiment_df["pos_count"] +
sentiment_df["neg_count"]).replace(0, pd.NA)
print("\n 年度 MD&A 情感结果： ")
print(sentiment_df)
# =====
# 5) 保存结果（可选）
# =====
out_csv = MDNA_DIR / "mdna_sentiment_by_year.csv"
sentiment_df.to_csv(out_csv, index=False, encoding="utf-8-sig")
print("\n 已保存： ", out_csv)

```

6. 画出情感的折线图

使用代码画出情感折线图，代码如下，下图展示了 2020-2024 年贵州茅台 MD&A 文本的情感指数变化趋势。

```

# 情感指数折线图
plt.figure(figsize=(8, 4))
plt.plot(sentiment_df["year"], sentiment_df["sentiment"], marker="o")
plt.axhline(0, linestyle="--")
plt.xlabel("年份")
plt.ylabel("MD&A 情感指数")
plt.title("管理层讨论与分析（MD&A）情感变化")
plt.tight_layout()
plt.show()

```

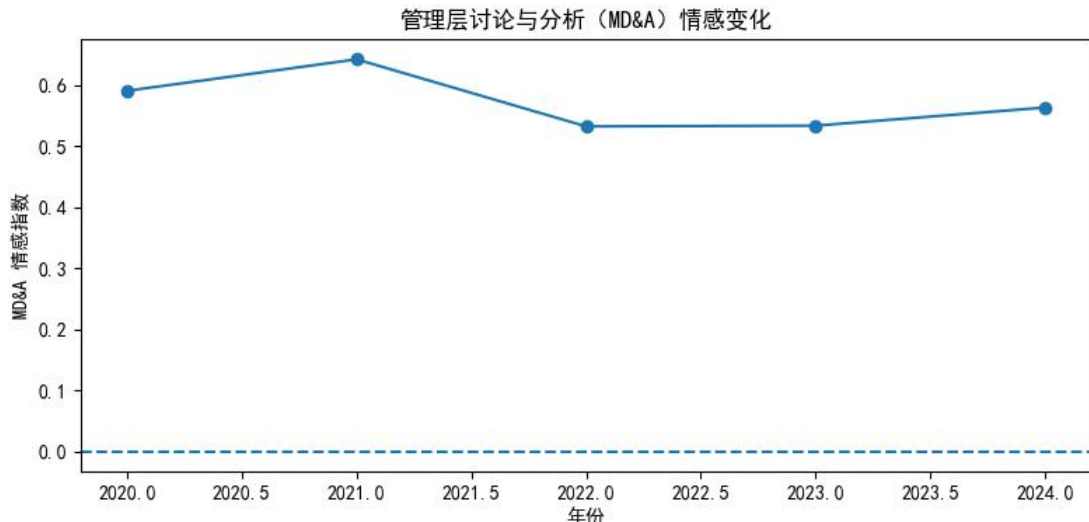


图 5：贵州茅台 2020-2024 年 MD&A 情感指数

（1）整体情绪乐观：五年间的情感指数始终为正，反映出管理层对公司经营状况和发展前景持积极、乐观的态度。

（2）2022 年情绪低点：2022 年情感指数出现明显回落，这与当年复杂的外部环境（如疫情反复、经济下行压力加大）相符。我们在该年份的 MD&A 文本中找到了相关证据，例如：“...面临不少难题...”、“...攻坚克难...”等表述增多，同时关于“风险”和“不确定性”的讨论也更为频繁。

六、整体分析

1. 先根据词云图的展示，结合公司的发展战略进行描述

词云图所揭示的高频词完美印证了贵州茅台公开的战略表述。公司坚定不移地走高质量发展之路，将“质量”视为生命线，并通过持续的“创新”和“改革”来驱动“发展”。无论是对“市场”的精耕细作，还是对内部“管理”的精益求精，都服务于一个核心目标——巩固和提升其在全球烈酒市场的领导地位。

2. 将以上的股票数据、财务数据、宏观数据、情感分析的结果在一起进行总结

综合所有维度的分析，我们可以得出以下结论：

贵州茅台是一家具备卓越投资价值的“时间的朋友”。

从市场表现看，其股价虽有波动，但长期趋势向上，风险收益比极佳，是核心资产配置的首选。

从财务绩效看，公司展现出无与伦比的盈利能力、极其稳健的财务结构和充沛健康的现金流，构成了其价值的坚实内核。

从宏观联动看，贵州茅台表现出强大的抗周期属性，其业绩增长与宏观经济的相关性较弱，更多由自身品牌力和稀缺性驱动，是穿越牛熊的优质标的。

从文本情感看，管理层始终传递出积极信号，即便在困难年份也展现出坚定的信心和清晰的战略定力。

综上所述，贵州茅台凭借其深厚的品牌护城河、卓越的商业模式、稳健的财务状况和优秀的公司治理，在未来相当长的时间内，仍将是资本市场中极具吸引力的价值投资标的。对于追求长期稳健回报的投资者而言，贵州茅台无疑是值得重点关注和持有的核心资产。